



Security Assessment Report

Orca Whirlpools

PR 95, 94 and 96

January 14, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the PRs of the Orca Whirlpools program.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 3 issues or questions.

#	Task	Type	Commit
P1	PR95: Pinocchio inc & dec liq ops	Solana	08b4336 (11/11/2025) → cc6953e (11/19/2025)
P2	PR94: Add reposition liquidity instruction	Solana	09a137f (11/14/2025) → f469113 (11/19/2025)
P3	PR96: Add Increase Liquidity By Token Amounts Ix	Solana	45b117f (11/19/2025) → bee1679 (11/23/2025)

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview 3

Findings in Detail 4

 [P2-I-01] Early rent check may cause unnecessary top-up 4

 [P3-L-01] Division-by-zero risk in estimate_max_liquidity_from_token_amounts 6

 [P3-I-01] Redundant tick order check 7

Appendix: Methodology and Scope of Work 8

Result Overview

Issue	Impact	Status
PR95: PINOCCHIO INC & DEC LIQ OPS		
No issues found		
PR94: ADD REPOSITION LIQUIDITY INSTRUCTION		
[P2-I-01] Early rent check may cause unnecessary top-up	Info	Resolved
PR96: ADD INCREASE LIQUIDITY BY TOKEN AMOUNTS IX		
[P3-L-01] Division-by-zero risk in estimate_max_liquidity_from_token_amounts	Low	Resolved
[P3-I-01] Redundant tick order check	Info	Resolved

Findings in Detail

PR94: ADD REPOSITION LIQUIDITY INSTRUCTION

[P2-I-01] Early rent check may cause unnecessary top-up

Identified in commit [f469113](#).

The PR introduces `reposition_liquidity_v2`, which aims to atomically execute `decrease_liquidity_v2` + `reset_position_range` + `increase_liquidity_v2`.

In the original `reset_position_range`, the program uses `ensure_position_has_enough_rent_for_ticks` to check whether the position has enough rent to increase the size for two ticks. If not, it transfers lamports from the funder to the position via the transfer CPI call.

```
/* programs/whirlpool/src/instructions/reset_position_range.rs */
063 | let position_rent_required = rent.minimum_balance(Position::LEN);
064 | let tick_required_rent = get_tick_rent_amount()? * 2;
065 | let all_required_rent = position_rent_required
066 |     .checked_add(tick_required_rent)
067 |     .ok_or(crate::errors::ErrorCode::RentCalculationError)?;
068 |
069 | let position_lamports = position.to_account_info().lamports();
070 | if position_lamports < all_required_rent {
```

In `reposition_liquidity_v2`, this rent check is performed before `decrease_liquidity_from_existing_range`.

```
/* programs/whirlpool/src/pinocchio/instructions/reposition_liquidity_v2.rs */
122 | pino_ensure_position_has_enough_rent_for_ticks(
123 |     funder_info,
124 |     position_account_info,
125 |     system_program_info,
126 | )?;
150 | decrease_liquidity_from_existing_range(
151 |     whirlpool_info.key(),
152 |     &mut whirlpool,
153 |     &mut position,
154 |     position_account_info,
155 |     existing_tick_array_lower_info,
156 |     existing_tick_array_upper_info,
157 |     timestamp,
158 |     &mut existing_range_token_a_decrease_amount,
159 |     &mut existing_range_token_b_decrease_amount,
160 | )?;
181 | position.reset_position_range(
182 |     &whirlpool,
183 |     data.new_tick_lower_index,
184 |     data.new_tick_upper_index,
```

```
185 |         true,  
186 |     )?;
```

If `decrease_liquidity_from_existing_range` shrinks the tick array size, some lamports are transferred from the tick array back to the position. Because the rent check happens before this shrink, the funder may be forced to top up the position unnecessarily.

It's recommended to move the rent check to after `decrease_liquidity_from_existing_range`, and before `position.reset_position_range`.

Resolution

This issue has been fixed by [9df8538](#).

PR96: ADD INCREASE LIQUIDITY BY TOKEN AMOUNTS IX

[P3-L-01] Division-by-zero risk in estimate_max_liquidity_from_token_amounts

Identified in commit [bee1679](#).

The `estimate_max_liquidity_from_token_amounts` function is used to estimate the maximum liquidity that can be added given a certain amount of tokens.

In the current implementation, if the input `current_sqrt_price` equals `lower_sqrt_price` (the pool price sits exactly on the lower tick), there is a division-by-zero risk:

```
/* programs/whirlpool/src/math/token_math.rs */
362 | let lower_sqrt_price = sqrt_price_from_tick_index(tick_lower_index);
363 | let upper_sqrt_price = sqrt_price_from_tick_index(tick_upper_index);
365 | if current_sqrt_price >= upper_sqrt_price {
367 |     est_liquidity_for_token_b(upper_sqrt_price, lower_sqrt_price, token_max_b)
368 | } else if current_sqrt_price < lower_sqrt_price {
370 |     est_liquidity_for_token_a(lower_sqrt_price, upper_sqrt_price, token_max_a)
371 | } else {
373 |     let liq_a = est_liquidity_for_token_a(current_sqrt_price, upper_sqrt_price, token_max_a)?;
374 |     let liq_b = est_liquidity_for_token_b(current_sqrt_price, lower_sqrt_price, token_max_b)?; // @audit:
    ↪ divide by zero if current_sqrt_price == lower_sqrt_price
375 |     Ok(liq_a.min(liq_b))
376 | }

/* programs/whirlpool/src/math/token_math.rs */
394 | fn est_liquidity_for_token_b(
395 |     sqrt_price_0: u128,
396 |     sqrt_price_1: u128,
397 |     token_amount_b: u64,
398 | ) -> Result<u128, ErrorCode> {
399 |     let (sqrt_price_lower, sqrt_price_upper) = increasing_price_order(sqrt_price_0, sqrt_price_1);
400 |     let sqrt_price_diff = sqrt_price_upper - sqrt_price_lower; // @audit: divide by zero if
    ↪ current_sqrt_price == lower_sqrt_price
401 |     let numerator_x64: u128 = (token_amount_b as u128) << 64;
402 |     let liquidity_u128 = numerator_x64 / sqrt_price_diff;
403 |     Ok(liquidity_u128)
404 | }
```

Recommendation. Change `current_sqrt_price < lower_sqrt_price` to `current_sqrt_price <= lower_sqrt_price`.

Resolution

This issue has been fixed by [03dc489](#).

PR96: ADD INCREASE LIQUIDITY BY TOKEN AMOUNTS IX [P3-I-01] Redundant tick order check

Identified in commit [bee1679](#).

In the `increase_liquidity_by_token_amounts_v2` instruction, the program checks whether `position.tick_upper_index() < position.tick_lower_index()`.

```
/* programs/whirlpool/src/pinocchio/instructions/increase_liquidity_by_token_amounts_v2.rs */
109 | if position.tick_upper_index() < position.tick_lower_index() {
110 |     return Err(WhirlpoolErrorCode::InvalidTickIndex.into());
111 | }
```

However, during the position opening process, the protocol already enforces that the upper tick index is greater than the lower tick index. As a result, this check is redundant.

```
/* programs/whirlpool/src/state/position.rs */
123 | fn validate_tick_range_for_whirlpool(
124 |     whirlpool: &Account<Whirlpool>,
125 |     tick_lower_index: i32,
126 |     tick_upper_index: i32,
127 | ) -> Result<()> {
128 |     if !Tick::check_is_usable_tick(tick_lower_index, whirlpool.tick_spacing)
129 |     || !Tick::check_is_usable_tick(tick_upper_index, whirlpool.tick_spacing)
130 |     || tick_lower_index >= tick_upper_index
131 |     {
132 |         return Err(ErrorCode::InvalidTickIndex.into());
133 |     }
134 |
135 |     // On tick spacing >= 2^15, should only be able to open full range positions
136 |     if whirlpool.tick_spacing >= FULL_RANGE_ONLY_TICK_SPACING_THRESHOLD {
137 |         let (full_range_lower_index, full_range_upper_index) =
138 |             Tick::full_range_indexes(whirlpool.tick_spacing);
139 |         if tick_lower_index != full_range_lower_index || tick_upper_index != full_range_upper_index
140 |         {
141 |             return Err(ErrorCode::FullRangeOnlyPool.into());
142 |         }
143 |     }
144 |
145 |     Ok(())
146 | }
```

Resolution

This issue has been fixed by [b62957d](#).

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Orca Management Company, S.A (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

